北京郵電大學

Beijing University of Posts and Telecommunications

BUPT Compilers Lab6 2024Fall

Name: 项 枫

Student ID: 2022211570

Class Number: 2022211801

Container Number:

6eca9c2b53e9aaa0c65b6af6613814be05cc0d8ccc63b7df064c4a7230aab09f

1 Code

1.1 json.h

Json's data structure is implemented in json.h.

1.1.1 JsonValue is the basic data structure of Json. It contains a JsonType and a JsonUnion.

```
struct JsonValue {
    JsonType type;
    JsonUnion value;
};
```

1.1.2 JsonType is an enumeration of all possible types of Json.

```
typedef enum {
    JSON_NULL,
    JSON_BOOLEAN,
    JSON_INTEGER,
    JSON_NUMBER,
    JSON_STRING,
    JSON_ARRAY,
    JSON_OBJECT
} JsonType;
```

1.1.3 JsonUnion is a union of all possible values of Json.

```
typedef union {
    int boolean;
    long long integer;
    double number;
    char *string;
    JsonArray array;
    JsonObject object;
} JsonUnion;
```

1.1.4 sonArray is a dynamic array of JsonValues.

```
typedef struct {
    size_t size;
    size_t capacity;
    JsonValue *values;
} JsonArray;
```

1.1.5 JsonObjectMember is a key-value pair of JsonValues.

```
typedef struct {
    char *key;
    JsonValue *value;
} JsonObjectMember;
```

1.1.6 JsonObject is a dynamic array of JsonObjectMembers.

```
typedef struct {
    size_t size;
    size_t capacity;
    JsonObjectMember *members;
} JsonObject;
```

1.2 json.c

json.c as follows.

```
#include "json.h"
#include <stdio.h>

void json_print(JsonValue json) {
    switch (json.type) {
        case JSON_NULL:
            printf("null");
            break;
        case JSON_BOOLEAN:
            printf(json.value.boolean ? "true" : "false");
            break;
        case JSON_INTEGER:
            printf("%lld", json.value.integer);
            break;
        case JSON_NUMBER:
            printf("%g", json.value.number);
            break;
        case JSON_STRING:
            printf("\"%s\"", json.value.string);
            break;
        case JSON_ARRAY:
            printf("[");
            for (size_t i = 0; i < json.value.array.size; i++) {
                if (i > 0) {
                    printf(", ");
                }
                json_print(json.value.array.values[i]);
            }
            printf("]");
            break;
        case JSON_OBJECT:
            printf("{");
            for (size_t i = 0; i < json.value.object.size; i++) {
                if (i > 0) {
```

```
                    printf(", ");
                }
                printf("\"%s\": ", json.value.object.members[i].key);
                json_print(*json.value.object.members[i].value);
            }
            printf("}");
            break;
        default:
            break;
    }
}
```

1.3 main.c

The availability of Json's data structure is tested in main.c as follows.

```
#include <stdio.h>
#include <stdlib.h>
#include "json.h"

int main() {
    // Create a JSON object
    JsonValue json;
    json.type = JSON_OBJECT;
    json.value.object.size = 0;
    json.value.object.capacity = 10;
    // Allocate memory for members
    json.value.object.members = malloc(sizeof(JsonObjectMember) *
json.value.object.capacity);
    // Add a member of type string
    json.value.object.members[json.value.object.size].key = "Name";
    json.value.object.members[json.value.object.size].value = malloc(sizeof(JsonValue));
    json.value.object.members[json.value.object.size].value->type = JSON_STRING;
    json.value.object.members[json.value.object.size].value->value.string = "项枫";
    json.value.object.size++;
    // Add a member of type integer
    json.value.object.members[json.value.object.size].key = "Student ID";
    json.value.object.members[json.value.object.size].value = malloc(sizeof(JsonValue));
    json.value.object.members[json.value.object.size].value->type = JSON_INTEGER;
    json.value.object.members[json.value.object.size].value->value.integer = 2022211570;
    json.value.object.size++;
    // Add another member of type integer
    json.value.object.members[json.value.object.size].key = "Class";
    json.value.object.members[json.value.object.size].value = malloc(sizeof(JsonValue));
    json.value.object.members[json.value.object.size].value->type = JSON_INTEGER;
    json.value.object.members[json.value.object.size].value->value.integer = 2022211801;
```

```
    json.value.object.size++;
    // Print the JSON object
    json_print(json);
    puts("");
    return 0;
}
```

2 Run

These commands were used:

```
make lltest
```

```
./json
```

The screenshot of the compilation and execution results.